

Product Return Prediction — Capstone Exercises

Machine Learning Crash Course · Instructor Edition (with answer key)

These are not "train a model" tasks — they are **investigations**. For each one, write a hypothesis first, then let the data confirm or refute it. The dataset has real traps built in, so the goal is to build **habits of rigour**: verify before you transform, question every "important" feature, and choose metrics that match the business. An answer key for instructors is on the last page.

Tier 1 · Warm-up: EDA & Data Quality

Exercise 1 **EASY** The Bug Hunt

Before any modelling, find **at least three** data-quality problems in `returns_dataset.csv`. For each, describe the problem and how you would fix it.

You'll practise: real data is dirty; inspect categorical value counts, missing values, and distributions before trusting a single number.

Deliverable: a short table of issue → evidence → fix.

Exercise 2 **EASY** Guess-then-Check

Without looking at the data, write down the **three features you believe most predict a return**. Now compute each feature's relationship with `returned` (correlation for numeric, return-rate-by-group for categorical). How many of your guesses survived?

You'll practise: intuition is a starting point, not evidence. Some "obvious" drivers turn out to be noise.

Deliverable: your 3 guesses + a ranked table of what actually matters.

Tier 2 · Core: Feature Engineering & Modelling

Exercise 3 **CORE** Should You Log-Transform It?

For every numeric column, decide whether it needs a `log` transform. **Justify each decision with a number** (e.g. skewness), not a guess. Then check: does logging a non-skewed column help, hurt, or do nothing?

You'll practise: transforms are earned, not applied by reflex. Measure skew first.

Deliverable: a table of column → skew → transform? → reason.

Exercise 4 **CORE** The Redundant-Feature Trap

Engineer `total_support_contacts = customer_support_calls + chat_interactions`, but keep **all three** columns. Train Logistic Regression and inspect the coefficients and the design-matrix condition number. What goes wrong, and how do you fix it?

You'll practise: spotting multicollinearity — when one feature is an exact combination of others, coefficients become unstable and uninterpretable.

Deliverable: the condition number before/after your fix + a one-line diagnosis.

Exercise 5 CORE The Honest Baseline

Build a model that **always predicts "not returned."** Report its Accuracy, Precision, and Recall. Then argue which single metric this business should optimise, and why.

You'll practise: why Accuracy misleads on imbalanced data, and how to pick a metric from the cost of each error.

Deliverable: the baseline's metrics + a 3-sentence business argument.

Tier 3 · Stretch: Interpretation & Rigour

Exercise 6 STRETCH ★ Debunk the Feature Importance

The feature with the **largest coefficient magnitude** is a single brand dummy, `brand_Brand_5`. Does that mean `Brand_5` orders are highly likely to be returned? Investigate the coefficient's *sign*, compare it to `Brand_5`'s real return rate, and explain why the number is so large. Propose a fix and re-run.

You'll practise: the most misused idea in applied ML — "importance" is not "cause." You'll meet sign vs magnitude, adjusted vs raw effects, and double-encoding.

Deliverable: a paragraph explaining the trap + before/after coefficients.

Exercise 7 STRETCH Hunt for Leakage

The task is to predict a return **before the order ships**. Go through the feature list and flag any column that (a) might not be known at prediction time, or (b) could leak the answer. What would you do about the strongest such features?

You'll practise: target leakage and deployment realism — the most "powerful" features are sometimes powerful because they cheat.

Deliverable: a flagged feature list with a keep / drop / caution decision each.

Exercise 8 STRETCH Tune the Threshold

The model uses a 0.5 probability cutoff. Move the threshold to achieve **90% recall**. Report what happens to precision, and decide whether that trade is worth it for this business.

You'll practise: the decision threshold is a business lever, not a fixed default.

Deliverable: a precision-recall trade-off note + your recommended threshold.

Capstone Challenge · Beat the Baseline, Honestly

A clean Logistic Regression reaches roughly **F1 $\approx$ 0.78**. Try to beat it **without leakage** — and be ready to **defend every feature you keep or add**. You are graded on the quality of your justification, not just

the score. Warning: adding more features often does *not* help.

Instructor Answer Key

Expected findings · not for student distribution

Ex 1 — Bug Hunt. (1) `fulfillment_type` has a casing/whitespace duplicate — "marketplace fulfilled" (60 rows) vs "Marketplace Fulfilled" — which silently splits one category into two. (2) Missing values: `product_rating` 240, `seller_rating` 180, `review_count` 120, `shipping_distance_km` 120. (3) `product_price` is heavily right-skewed with an extreme max (~64,879 vs median ~597).

Ex 2 — Guess-then-Check. Common guesses (price, discount) are **near-zero** predictors here (corr ≈ 0.00 and 0.01; every product category sits at ~30% returns). The real drivers: `fragile_item` (54% vs 25% return rate), `delayed_delivery` (53% vs 25%), and payment method (COD 50.5% vs UPI 16%), plus historical brand/product return-rate columns.

Ex 3 — Log transform. Only `product_price` is genuinely skewed (skew ≈ 16). `review_count` (0.01) and `product_page_views` (0.08) are symmetric; `shipping_distance_km` (≈ 0.00) is essentially uniform. Logging shipping distance actually **weakens** its signal (corr 0.162 → 0.146). Lesson: verify skew first; transform only price.

Ex 4 — Redundant feature. With all three columns the numeric design matrix is rank-deficient — condition number jumps to ~10¹⁵ (vs ~3.3 when fixed), and the support signal splits arbitrarily across three coefficients. F1 is unchanged (~0.78) because L2 regularisation hides it. Fix: keep `total_support_contacts` only (or the two components only), never all three.

Ex 5 — Honest baseline. "Always not-returned" scores ~70% **accuracy** (the class balance) while catching **zero** returns (Recall = 0, Precision undefined/0). Because each missed return carries the full reverse-logistics cost, the business should prioritise **Recall**, with Precision watched so preventive actions aren't wasted; F1 summarises the balance.

Ex 6 — Debunk importance. ★ `brand_Brand_5`'s coefficient is **-3.02** — strongly *negative*. `Brand_5` has the **lowest** real return rate (~3.6%), so it predicts "not returned," the opposite of the naive reading. The magnitude is inflated because **brand is double-encoded**: the 30 one-hot `brand_*` dummies *and* the numeric `brand_return_rate`. Dropping the dummies restores a healthy fit and lifts `brand_return_rate`'s coefficient from ~0.36 to ~1.33, with F1 essentially unchanged. Lessons: check the sign; a coefficient is an *adjusted* association, not a cause; don't feed the same information twice.

Ex 7 — Leakage. The historical `*_return_rate` columns (product/brand especially) are the strongest predictors but risk **target leakage** if they were computed using the current order's outcome — students should ask whether the value would truly be available at prediction time. Also, `delayed_delivery`, `customer_support_calls`, and `chat_interactions` are arguably **post-purchase**, which conflicts with the "predict before shipping" framing — a great discussion point.

Ex 8 — Threshold. Lowering the cutoff below 0.5 raises Recall toward 90% but drops Precision (more false alarms). There is no single right answer — the point is that students choose the threshold from the relative cost of a missed return vs a wasted preventive action, not from the default.

Capstone. The honest baseline is $F1 \approx 0.78$. In practice, **removing** redundant/low-signal features (the price-math features, the diluting `avg_return_rate`, the duplicate brand encoding) held or slightly **improved** F1 while making the model interpretable. Reward students who justify *subtraction*, not just addition, and who avoid leakage.